

FACET-SNP-307 — Desenvolvimento Web

Turma 2026.1 · Carga horária: 60h

Aula 11/05/2026

Introdução ao Backend com Bun + Express

Duração: 3 horas (19h–22h)
Professor: Ivan Luiz Pedroso Pires
Projeto-tema: Sistema de Empréstimo de Equipamentos do Laboratório

1. Objetivos de aprendizagem

Ao final desta aula, você será capaz de:

- Compreender o papel do **backend** em uma aplicação web e sua comunicação com o frontend
- Instalar e utilizar o **Bun** como runtime JavaScript no servidor
- Criar um servidor HTTP utilizando o framework **Express**
- Implementar uma **API REST** que retorna dados em formato JSON
- Utilizar **query strings** para receber e processar filtros no backend
- Integrar um **formulário HTML** com o backend via `fetch`

2. Pré-requisitos

- Notebook com Wi-Fi funcionando
- Editor de código instalado (recomendado: **VS Code**)
- Navegador atualizado (Chrome, Firefox ou Edge)
- Conhecimento de HTML, CSS, JavaScript, formulários, `fetch` e JSON (já trabalhados nas aulas anteriores)

3. Conceitos fundamentais

3.1. A arquitetura cliente-servidor

Uma aplicação web moderna é composta por **duas partes** que se comunicam:

- **Frontend (cliente):** é o que roda no navegador — HTML, CSS e JavaScript que você já estudou. É a parte visível.
- **Backend (servidor):** é o programa que fica em execução em um computador remoto, recebendo requisições, processando dados e devolvendo respostas. É invisível ao usuário, mas é onde mora a lógica do negócio.

A comunicação entre as duas partes acontece pelo protocolo **HTTP**, normalmente em formato **JSON** nas APIs modernas. Quando você faz `fetch('/equipamentos')` no frontend, alguém precisa estar do outro lado da linha para responder. Esse "alguém" é o backend que vamos construir hoje.

3.2. Runtime e framework

Para executar JavaScript fora do navegador, precisamos de um **runtime**. Os principais são:

- **Node.js** (2009) — o pioneiro, padrão histórico do mercado
- **Bun** (2022) — runtime moderno, mais rápido, com gerenciador de pacotes integrado

Vamos usar o **Bun** por ser mais rápido, simples de instalar e por já incluir gerenciador de pacotes, bundler e test runner em um único binário.

Sobre o framework HTTP: o Bun (e o Node) traz APIs básicas para criar servidores, mas escrever tudo manualmente é trabalhoso. Por isso usamos o **Express**, framework minimalista que simplifica o tratamento de rotas, requisições e respostas.

✦ PARA SABER MAIS

Bun e Node convivem sem problemas no mesmo computador. Tutoriais que você encontrar na internet com `npm install` e `node app.js` também funcionam — basta traduzir os comandos para `bun add` e `bun app.js`.

3.3. API REST e JSON

Uma **API REST** é uma forma padronizada de organizar endpoints HTTP. Cada recurso (ex: equipamentos) é exposto em uma URL e manipulado por verbos HTTP:

Verbo HTTP	Significado	Exemplo
GET	Buscar dados	GET /equipamentos
POST	Criar novo recurso	POST /equipamentos
PUT	Atualizar recurso existente	PUT /equipamentos/1
DELETE	Remover recurso	DELETE /equipamentos/1

O formato de troca é o **JSON** — o mesmo que vocês já conhecem do frontend.

3.4. Query strings

Quando precisamos enviar **filtros** ou **parâmetros opcionais** em uma requisição GET, usamos query strings:

```
GET /equipamentos?status=disponivel&tipo=projektor
```

No backend, conseguimos ler esses valores através de `req.query`:

```
app.get('/equipamentos', (req, res) => {  
  const { status, tipo } = req.query;  
  // status = 'disponivel'  
  // tipo = 'projektor'  
});
```

4. Passo a passo: construindo a API

1 Instalar o Bun

Acesse <https://bun.sh> e siga as instruções para o seu sistema operacional.

Linux/macOS:

```
curl -fsSL https://bun.sh/install | bash
```

Windows (PowerShell):

```
powershell -c "irm bun.sh/install.ps1 | iex"
```

Verifique a instalação:

```
bun --version
```

2 Criar o projeto

```
mkdir lab-empresta  
cd lab-empresta  
bun init -y
```

Depois, apague os arquivos `index.ts` e `tsconfig.json` gerados (usaremos JavaScript puro).

3 Configurar ES Modules

Edite o arquivo `package.json` e adicione a chave `"type": "module"`:

```
{  
  "name": "lab-empresta",  
  "type": "module",  
  "scripts": {  
    "dev": "bun --hot server.js"  
  }  
}
```

4 Instalar o Express

```
bun add express
```

5 Criar o servidor

Crie o arquivo `server.js` com o seguinte conteúdo:

```
import express from 'express';

const app = express();

// Middleware para receber JSON no corpo da requisição
app.use(express.json());

// Dados estáticos (na próxima aula vamos trocar isso por um banco de dados)
const equipamentos = [
  { id: 1, nome: 'Projektor Epson X05+', tipo: 'projektor', status: 'disponivel' },
  { id: 2, nome: 'Notebook Dell Latitude', tipo: 'notebook', status: 'emprestado' },
  { id: 3, nome: 'Kit Arduino Uno', tipo: 'kit', status: 'disponivel' },
  { id: 4, nome: 'Cabo HDMI 5m', tipo: 'cabo', status: 'disponivel' },
  { id: 5, nome: 'Webcam Logitech C920', tipo: 'webcam', status: 'manutencao' },
];

// Rota de boas-vindas
app.get('/', (req, res) => {
  res.send('API do Sistema de Empréstimo rodando!');
});

// Rota que retorna todos os equipamentos
app.get('/equipamentos', (req, res) => {
  res.json(equipamentos);
});

// Inicia o servidor
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

6 Executar o servidor

```
bun run dev
```

Abra o navegador em **http://localhost:3000** e depois em **http://localhost:3000/equipamentos**. Você verá os dados em JSON.

7 Adicionar filtros via query string

Modifique a rota `/equipamentos` para aceitar filtros:

```
app.get('/equipamentos', (req, res) => {
  const { status, tipo, busca } = req.query;

  let resultado = equipamentos;

  if (status) {
    resultado = resultado.filter(e => e.status === status);
  }

  if (tipo) {
    resultado = resultado.filter(e => e.tipo === tipo);
  }

  if (busca) {
    const termo = busca.toLowerCase();
    resultado = resultado.filter(e => e.nome.toLowerCase().includes(termo));
  }

  res.json({
    total: resultado.length,
    equipamentos: resultado
  });
});
```

Teste no navegador:

- <http://localhost:3000/equipamentos?status=disponivel>
- <http://localhost:3000/equipamentos?tipo=projedor>
- <http://localhost:3000/equipamentos?busca=dell>
- <http://localhost:3000/equipamentos?status=disponivel&tipo=kit>

DICA

O Bun com a flag `--hot` reinicia o servidor automaticamente quando você salva o arquivo. Não precisa parar e iniciar a cada mudança.

ATENÇÃO: LIMITAÇÃO DOS DADOS EM MEMÓRIA

Como os dados estão em um array no código, eles se perdem cada vez que o servidor é reiniciado. Na próxima aula vamos resolver isso usando um banco de dados de verdade.

5. Comandos de referência

Tarefa	Comando Bun	Equivalente Node/npm
Iniciar projeto	<code>bun init</code>	<code>npm init</code>
Instalar pacote	<code>bun add <pacote></code>	<code>npm install <pacote></code>
Instalar tudo	<code>bun install</code>	<code>npm install</code>
Rodar script	<code>bun run dev</code>	<code>npm run dev</code>
Rodar arquivo	<code>bun server.js</code>	<code>node server.js</code>
Hot reload	<code>bun --hot server.js</code>	<code>nodemon server.js</code>

6. Códigos HTTP úteis

Código	Significado	Quando usar
200	OK	Sucesso em uma operação GET, PUT
201	Created	Recurso criado com sucesso (POST)
204	No Content	Sucesso sem conteúdo de retorno (DELETE)
400	Bad Request	Dados enviados pelo cliente estão errados
404	Not Found	Recurso buscado não existe
500	Internal Server Error	Erro no servidor (falha na lógica do backend)

Para retornar um código diferente de 200 no Express:

```
res.status(404).json({ erro: 'Equipamento não encontrado' });
```

🔗 Exercício Avaliativo

Contexto

O laboratório de informática do curso de Sistemas de Informação precisa de uma **interface de consulta** para que professores e alunos possam verificar quais equipamentos estão disponíveis antes de ir ao laboratório. A interface deve permitir filtrar a lista por status, tipo e nome.

Sua tarefa é construir um sistema completo composto por um **backend** (API que retorna dados filtrados) e um **frontend** (página HTML com formulário de busca).

Requisitos do Backend (parte que vimos em aula)

1. Servidor Express rodando em `http://localhost:3000`
2. Array com **no mínimo 10 equipamentos**, cada um com: `id`, `nome`, `tipo`, `status` e `descricao`
3. Tipos sugeridos: `projektor`, `notebook`, `kit`, `cabo`, `webcam`, `microfone`
4. Status possíveis: `disponivel`, `emprestado`, `manutencao`
5. Endpoint `GET /equipamentos` aceitando os filtros opcionais:
 - `?status=...` — filtra pelo status
 - `?tipo=...` — filtra pelo tipo
 - `?busca=...` — busca textual no nome (case-insensitive)
6. Filtros devem poder ser **combinados** (ex: status + tipo ao mesmo tempo)
7. A resposta deve retornar um JSON com `total` e `equipamentos`

Requisitos do Frontend (parte sua)

1. Página HTML chamada `index.html` com um formulário contendo:
 - Um `<select>` para o filtro de **status** (com opção "Todos" + os 3 status)
 - Um `<select>` para o filtro de **tipo** (com opção "Todos" + os tipos disponíveis)
 - Um `<input type="text">` para a busca textual
 - Um `<button>` para disparar a busca
2. Ao clicar no botão, o JavaScript deve:
 - Ler os valores dos campos
 - Montar a URL com query string apropriada (lembrando: campos vazios ou "Todos" não devem entrar na URL)
 - Usar `fetch` para chamar a API
 - Renderizar os equipamentos retornados em uma lista ou tabela
 - Exibir o **total** de resultados encontrados
3. CSS básico para não ficar feio (use o que vocês quiserem, inclusive frameworks)

Bônus (opcional, vale ponto extra)

- Buscar em tempo real (sem botão), com `debounce` de 300ms
- Botão "Limpar filtros" que reseta o formulário e mostra todos
- Indicador visual de carregamento durante o `fetch`
- Tratamento de erro caso o servidor não esteja rodando

Entrega

- **Formato:** repositório no GitHub (público) ou pasta zipada
- **Estrutura mínima:**

```
lab-empresta/
├── server.js
├── package.json
├── public/
│   ├── index.html
│   ├── style.css
│   └── app.js
└── README.md ← instruções de como rodar
```

- **Prazo:** até a próxima aula (18/05/2026)
- **Como entregar:** link do repositório ou arquivo zip pelo canal combinado em aula

Critérios de avaliação

Item	Pontos
Backend funcionando com 10+ equipamentos	2,0
Filtros funcionando isoladamente (status, tipo, busca)	2,0
Filtros combinados funcionando	1,0
Formulário HTML completo	1,5
Integração fetch + renderização dinâmica	2,0
Organização do código e README	1,5
Total	10,0

💡 DICA DE ORGANIZAÇÃO

Sirva o frontend (HTML/CSS/JS) diretamente pelo Express usando `app.use(express.static('public'))`. Assim você acessa tudo em `http://localhost:3000` e não precisa de dois servidores diferentes.

7. Próxima aula

Na próxima aula (18/05/2026) vamos resolver a principal limitação do que construímos hoje: a perda de dados quando o servidor reinicia. Vamos conectar nosso backend a um **banco de dados PostgreSQL** hospedado no **Supabase**, modelar as tabelas do projeto e fazer com que nossa API consulte dados reais.

Para se preparar:

- Crie uma conta gratuita em <https://supabase.com> usando seu email institucional
- Mantenha o projeto `lab-empresta` versionado (recomendamos Git/GitHub)
- Traga dúvidas do exercício para discutirmos no início da aula